

Program : Diploma in Information Technology	
Course Code : 3264	Course Title: Software Engineering
Semester : 3	Credits: 3
Course Category: Program Core	
Periods per week: 3(L:3 T:0 P:0)	Periods per semester: 45

Course Objectives:

- To provide an exposure regarding the methods and procedures of developing a reliable and cost-effective software that can scale up for large systems and which can be used for producing it in large scale within a small span of time and also in economic manner.
- To introduce the concepts of reliability, maintainability, Testability and reusability.
- To introduce the various stages of the software design procedures and applying it in other engineering designs.

Course Prerequisites:

Topic	Course code	Course name	Semester
General principles of Computation and Programming		Introduction to IT Systems	1

Course Outcomes:

On completion of the course, students will be able to:

CO _n	Description	Duration (Hours)	Cognitive Level
CO1	Demonstrate the ethics of software design process	11	Understanding
CO2	Outline the agile software using agile software development techniques	10	Understanding
CO3	Illustrate the requirements, modelling and design of software engineering	12	Understanding
CO4	Illustrate the design, implementation and testing of software	10	Understanding
	Series Test	2	

CO – PO Mapping:

Course Outcomes	PO1	PO2	PO3	PO4	PO5	PO6	PO7
CO1	2						
CO2	2						
CO3	2						
CO4	2						

3-Strongly mapped, 2-Moderately mapped, 1-Weakly mapped

Course Outline:

Module Outcomes	Description	Duration (Hours)	Cognitive Levels
CO1	Demonstrate the ethics of software design process		
M1.1	List the software Engineering ethics based on professional responsibility	1	Remembering
M1.2	Contrast the software process models	1	Understanding
M1.3	Summarize the waterfall model derived from engineering process models	1	Understanding
M1.4	Define incremental development based on software evolution	1	Remembering
M1.5	Illustrate integration and configuration in software reuse context	1	Understanding
M1.6	Define software specification by identifying constraints on system's operation and development	1	Remembering
M1.7	Explain software design and implementation considering various systems and software	1	Understanding
M1.8	List the software validation process with respect to testing	1	Remembering
M1.9	Explain evolution of software	1	Understanding
M1.10	Illustrate how prototyping can be used to anticipate changes	1	Understanding
M1.11	Summarize the Incremental delivery process in software development and relate process improvement approaches with quality	1	Understanding
Contents: Software Engineering Ethics-Software process models-waterfall model-incremental development-Integration and configuration. Process Activities- software specification-design and implementation-software validation-software evolution. Coping with change-Prototyping-Incremental delivery Process Improvement			

CO2	Outline the agile software using agile software development techniques		
M2.1	Distinguish agile and plan driven software development	1	Understanding
M2.2	Illustrate the principles of agile software development with respect to its success rate	1	Understanding
M2.3	Explain agile development techniques in comparison to extreme programming practice	1	Understanding
M2.4	Interpret the need of refactoring in agile software development	1	Understanding
M2.5	Illustrate the key features of testing based on XP	1	Understanding
M2.6	List the advantages of pair programming in the context of XP	1	Remembering
M2.7	Summarize the scrum approach in agile project management	1	Understanding
M2.8	Explain the problems in agile while using long-lifetime systems	2	Understanding
M2.9	Extend the applicability of agile for large systems and organizations	1	Understanding
	Series Test-I	1	
Contents: Agile and plan driven software- Agile Methods-Agile development techniques-Refactoring- Test first development-Pair programming Agile project Management-scrum Scaling agile methods- Practical problems-agile for large systems-agile across organizations			
CO3	Illustrate the requirements, modelling and design of software engineering		
M3.1	Classification of software system requirements into functional and non-functional requirements	2	Understanding
M3.2	Illustrate natural language specification	1	Understanding
M3.3	Explain the requirements validation techniques for customer satisfaction	1	Understanding
M3.4	Explain the context model of representing software systems	1	Understanding
M3.5	Illustrate the use-case modelling of interaction models	1	Understanding
M3.6	Demonstrate the generalization technique to manage complexity in structural models.	1	Understanding
M3.7	Extend the software development technique to model driven approach in behavioral model	1	Understanding
M3.8	Explain the three types of abstract system model in model driven architecture	1	Understanding

M3.9	Compare the functional and non-functional requirements of a system in the context of architectural design decisions	1	Understanding
M3.10	List the fundamental architectural views of a software system	1	Remembering
M3.11	Compare layered, repository and client-server architecture of software architectural patterns	1	Understanding

Contents:

Requirements Engineering- Functional and non-functional requirements-**Requirements Specification-**natural language specification-**Requirements Validation-Context Models-Interaction Models-**Use case modelling-**Structural Models-**Generalization-**Behavioral Models-**Model driven engineering
Model driven architecture-abstract system models
Architectural design decisions – Architectural Views-Architectural patterns-layered, repository and client-server architecture pattern

CO4	Illustrate the design, implementation and testing of software		
M 4.1	Relate the software system with the environment where it works-i.e., context and interaction	1	Understanding
M 4.2	Explain architectural design based on context and environment	1	Understanding
M 4.3	Distinguish between the UML based design models	1	Understanding
M 4.4	List the various elements of design patterns	1	Remembering
M 4.5	List the different implementation issues in software engineering	1	Remembering
M 4.6	Explain unit testing process of development testing	1	Understanding
M 4.7	Illustrate component testing process of development testing	1	Understanding
M 4.8	Explain the test-driven approach of development	1	Understanding
M 4.9	Summarize the performance testing in the context of performance and reliability	1	Understanding
M 4.10	List the different types of user testing	1	Remembering
	Series Test- II	1	

Contents:

Object-Oriented design using the UML - System context and interactions-**Architectural Design-Design Models-Design Patterns-Implementation Issues-Testing –** Development testing-unit testing-component testing- **Test driven development-Release Testing –** Performance testing **User Testing**

Text / Reference

T/R	Book Title/Author
T1	Software Engineering –Ian Sommerville – Pearson publication
R1	Fundamentals of software engineering –Rajib Mall
R2	Software Engineering-Roger S Pressman
R3	Software Engineering K. K. Aggarwal, Yogesh Singh
R4	Software Engineering concepts –Richard Fairley Tata Mc-Graw-Hill Edition

Online Resources

Sl.No	Website Link
1	www.pearsoned.co.in
2	https://www.geeksforgeeks.org/software-engineering/
3	https://www.tutorialspoint.com/software_engineering/index.htm
4	https://www.javatpoint.com/software-engineering-tutorial